

Ultimate Productivity App — Build Plan

Stack

Layer	Tech
Android UI	Kotlin + Jetpack Compose
Backend API	Rust + Axum
Database (server)	PostgreSQL
Database (local)	SQLite (Room)
ORM (Rust)	SQLx
Auth	JWT tokens
HTTP Client	Retrofit
Deployment	Docker + VPS

Phase 0: Project Scaffold & Auth (Weeks 1–2)

Everything needed before we can build features. Two projects, one database, working auth.

0.1 — Backend Project Setup

Create the Rust project:

```
backend/
├── Cargo.toml
├── .env                                # DATABASE_URL, JWT_SECRET
├── src/
│   ├── main.rs                        # Axum server entry point, router setup
│   ├── config.rs                      # Load env vars, app config struct
│   └── error.rs                       # AppError type, IntoResponse impl for consistent JSON
└── errors
    ├── db.rs                          # PgPool setup, connection helper
    ├── middleware/
    │   └── auth.rs                    # JWT extraction + validation middleware
    ├── models/
    │   ├── mod.rs
    │   └── user.rs                   # User struct, CreateUser, LoginUser
    ├── routes/
    │   ├── mod.rs                    # Merge all routers
    │   └── auth.rs                   # /auth/* handlers
    └── migrations/
        └── 001_create_users.sql
```

Cargo.toml dependencies:

- `axum` — web framework
- `tokio` (full features) — async runtime
- `sqlx` (postgres, runtime-tokio, tls-rustls, migrate) — database

- `serde` + `serde_json` — serialization
- `jsonwebtoken` — JWT encode/decode
- `argon2` — password hashing
- `dotenvy` — load `.env` file
- `uuid` (v4, `serde`) — primary keys
- `chrono` (`serde`) — timestamps
- `tower-http` (cors) — CORS middleware
- `tracing` + `tracing-subscriber` — logging

`main.rs` must:

1. Load `.env` with `dotenvy::dotenv()`
2. Init tracing subscriber
3. Create `PgPool` from `DATABASE_URL`
4. Run SQLx migrations on startup
5. Build the Axum router with all route groups merged
6. Add CORS layer (allow all origins in dev)
7. Bind to `0.0.0.0:8080` and serve

0.2 — Docker Compose for Dev Database

`docker-compose.dev.yml`:

```
services:
  db:
    image: postgres:16
    restart: unless-stopped
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: dev
      POSTGRES_PASSWORD: devpass
      POSTGRES_DB: productivity
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

`.env` for backend:

```
DATABASE_URL=postgres://dev:devpass@localhost:5432/productivity
JWT_SECRET=dev-secret-change-in-prod
```

0.3 — Users Table & Auth Endpoints

Migration `001_create_users.sql`:

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE users (
```

```
    id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    email       VARCHAR(255) UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Models (models/user.rs):

```
// Database row
struct User {
    id: Uuid,
    email: String,
    password_hash: String,
    created_at: DateTime<Utc>,
}

// Request: register
struct CreateUser {
    email: String,
    password: String,
}

// Request: login
struct LoginUser {
    email: String,
    password: String,
}

// Response: return to client (no password hash)
struct UserResponse {
    id: Uuid,
    email: String,
    created_at: DateTime<Utc>,
}

// JWT claims
struct Claims {
    sub: String,    // user id
    exp: usize,     // expiry timestamp
}

// Response: after login/register
struct AuthResponse {
    token: String,
    user: UserResponse,
}
```

Endpoints (routes/auth.rs):

Method	Path	Body	Response	Auth?
POST	/auth/register	{ email, password }	AuthResponse	No
POST	/auth/login	{ email, password }	AuthResponse	No
GET	/auth/me	—	UserResponse	Yes

POST /auth/register logic:

1. Validate email format (contains @, non-empty)
2. Validate password length (min 8 characters)
3. Check if email already exists → 409 Conflict if so
4. Hash password with `argon2::hash_encoded()`
5. Insert into `users` table
6. Generate JWT token (24h expiry) containing user ID in `sub` claim
7. Return `AuthResponse { token, user }`

POST /auth/login logic:

1. Query user by email → 401 if not found
2. Verify password with `argon2::verify_encoded()` → 401 if wrong
3. Generate JWT token (24h expiry)
4. Return `AuthResponse { token, user }`

GET /auth/me logic:

1. Auth middleware extracts and validates JWT from `Authorization: Bearer <token>` header
2. Decode token, extract `sub` (user ID)
3. Query user by ID → 401 if not found
4. Return `UserResponse`

Auth middleware (middleware/auth.rs):

- Axum extractor that reads `Authorization` header
- Strips `Bearer` prefix
- Decodes JWT using `JWT_SECRET`
- Validates expiry
- Returns `Claims` to the handler (or 401 response)

0.4 — Android Project Setup**Create the Android project in Android Studio:**

- Template: Empty Compose Activity
- Package name: `com.app.productivity`
- Min SDK: API 26 (Android 8.0)
- Target SDK: API 34

Project structure:

```

android/
├── app/
│   ├── build.gradle.kts
│   └── src/main/
│       ├── AndroidManifest.xml
│       ├── java/com/app/productivity/
│       │   ├── MainActivity.kt           # Single activity, hosts Compose
│       │   ├── ProductivityApp.kt      # Application class
│       │   ├── navigation/
│       │   │   └── AppNavigation.kt    # NavHost + route definitions
│       │   └── ui/
│       └── theme/

```



- ## 0.5 — Android Auth Flow

- Uses Jetpack DataStore (Preferences)
- `saveToken(token: String)` — stores JWT
- `getToken(): Flow<String?>` — reads JWT
- `clearToken()` — logout

- Base URL: `http://10.0.2.2:8080` (emulator → host localhost) or your machine's local IP for physical device
- `AuthInterceptor`: reads token from `TokenManager`, attaches `Authorization: Bearer <token>` header to every request

- `OkHttpClient` with `HttpLoggingInterceptor` (for debug builds) + `AuthInterceptor`
- Single `Retrofit` instance with `GsonConverterFactory`

ApiService.kt:

```
interface ApiService {
    @POST("auth/register")
    suspend fun register(@Body request: RegisterRequest): AuthResponse

    @POST("auth/login")
    suspend fun login(@Body request: LoginRequest): AuthResponse

    @GET("auth/me")
    suspend fun getMe(): UserResponse
}
```

AuthViewModel.kt:

- State: `isLoading`, `error`, `isLoggedIn`
- `login(email, password)` → call API → save token → navigate to dashboard
- `register(email, password)` → call API → save token → navigate to dashboard
- `checkAuth()` → read token from `DataStore` → call `/auth/me` → if valid, go to dashboard; if not, go to login
- `logout()` → clear token → navigate to login

LoginScreen.kt:

- Email text field
- Password text field (with visibility toggle)
- "Login" button
- "Don't have an account? Register" link → navigates to `RegisterScreen`
- Show error messages from `ViewModel`
- Show loading spinner when `isLoading`

RegisterScreen.kt:

- Email text field
- Password text field
- Confirm password text field (validate match client-side)
- "Register" button
- "Already have an account? Login" link
- Same error/loading handling

AppNavigation.kt:

- Routes: `login`, `register`, `dashboard`, `sleep`, `sessions`, `calendar`
- Start destination: check if token exists → `dashboard` or `login`
- Bottom navigation bar with 4 tabs: Dashboard, Sleep, Sessions, Calendar
- Bottom bar only visible on main screens (not auth screens)

0.6 — Verify Everything Works

At this point, confirm:

- `docker compose -f docker-compose.dev.yml up -d` starts Postgres

- `cargo run` starts the backend on port 8080
- `POST /auth/register` creates a user and returns a token
- `POST /auth/login` returns a token for valid credentials
- `GET /auth/me` with the token returns the user
- Android app launches with login screen
- Register from the app → lands on empty dashboard
- Login from the app → lands on empty dashboard
- Kill app and reopen → stays logged in (token persisted)

Phase 1: Sleep Tracking — End to End (Weeks 3–5)

1.1 — Sleep Backend: Database Tables

Migration `002_create_sleep_tables.sql`:

```
CREATE TABLE sleep_records (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  target_bedtime  TIME NOT NULL,
  target_wake_time TIME NOT NULL,
  actual_bedtime  TIMESTAMPTZ NOT NULL,
  actual_wake_time TIMESTAMPTZ NOT NULL,
  quality_rating SMALLINT NOT NULL CHECK (quality_rating BETWEEN 1 AND 5),
  phone_pickups  INT NOT NULL DEFAULT 0,
  total_phone_minutes INT,
  notes          TEXT,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_sleep_records_user_id ON sleep_records(user_id);
CREATE INDEX idx_sleep_records_actual_bedtime ON sleep_records(user_id, actual_bedtime);

CREATE TABLE phone_pickups (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  sleep_record_id  UUID REFERENCES sleep_records(id) ON DELETE SET NULL,
  session_id   UUID, -- FK added later when sessions table exists
  picked_up_at TIMESTAMPTZ NOT NULL,
  duration_seconds INT NOT NULL DEFAULT 0,
  app_category  VARCHAR(50),
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_phone_pickups_sleep ON phone_pickups(sleep_record_id);
```

1.2 — Sleep Backend: Models

`models/sleep.rs`:

```
// Database row
struct SleepRecord {
  id: Uuid,
  user_id: Uuid,
```

```

    target_bedtime: NaiveTime,
    target_wake_time: NaiveTime,
    actual_bedtime: DateTime<Utc>,
    actual_wake_time: DateTime<Utc>,
    quality_rating: i16,
    phone_pickups: i32,
    total_phone_minutes: Option<i32>,
    notes: Option<String>,
    created_at: DateTime<Utc>,
    updated_at: DateTime<Utc>,
}

// Request: create/update
struct CreateSleepRecord {
    target_bedtime: NaiveTime,
    target_wake_time: NaiveTime,
    actual_bedtime: DateTime<Utc>,
    actual_wake_time: DateTime<Utc>,
    quality_rating: i16,           // 1-5
    phone_pickups: i32,
    total_phone_minutes: Option<i32>,
    notes: Option<String>,
}

// Response: sleep stats
struct SleepStats {
    avg_duration_minutes: f64,
    avg_quality: f64,
    total_records: i64,
    sleep_debt_minutes: f64,      // target - actual over period
    avg_phone_pickups: f64,
    best_quality_day: Option<String>,
    worst_quality_day: Option<String>,
}

```

models/phone_pickup.rs:

```

struct PhonePickup {
    id: Uuid,
    user_id: Uuid,
    sleep_record_id: Option<Uuid>,
    session_id: Option<Uuid>,
    picked_up_at: DateTime<Utc>,
    duration_seconds: i32,
    app_category: Option<String>,
    created_at: DateTime<Utc>,
}

struct CreatePhonePickup {
    sleep_record_id: Option<Uuid>,
    session_id: Option<Uuid>,
    picked_up_at: DateTime<Utc>,
    duration_seconds: i32,
    app_category: Option<String>,
}

```

1.3 — Sleep Backend: API Endpoints

Add routes/sleep.rs:

Method	Path	Body / Params	Response	Auth?
POST	/sleep	CreateSleepRecord	SleepRecord	Yes
GET	/sleep	Query: ?start=DATE&end=DATE	Vec<SleepRecord>	Yes
GET	/sleep/:id	—	SleepRecord	Yes
PUT	/sleep/:id	CreateSleepRecord	SleepRecord	Yes
DELETE	/sleep/:id	—	204 No Content	Yes
GET	/sleep/stats	Query: ? range=week\ month\ custom&start=DATE&end=DATE	SleepStats	Yes

POST /sleep logic:

1. Extract user ID from auth middleware
2. Validate: `actual_bedtime < actual_wake_time`
3. Validate: `quality_rating` between 1 and 5
4. Validate: `phone_pickups >= 0`
5. Insert into `sleep_records`
6. Return the created record with 201

GET /sleep logic:

1. Extract user ID from auth middleware
2. Parse optional `start` and `end` date query params
3. Default: last 30 days if no params
4. Query `sleep_records` WHERE `user_id = $1` AND `actual_bedtime BETWEEN $2 AND $3` ORDER BY `actual_bedtime DESC`
5. Return array

GET /sleep/stats logic:

1. Parse `range` param: `week` = last 7 days, `month` = last 30 days, `custom` = use `start/end`
2. Query all records in range
3. Calculate:
 - `avg_duration_minutes`: AVG of (`actual_wake_time` - `actual_bedtime`) in minutes
 - `avg_quality`: AVG of `quality_rating`
 - `total_records`: COUNT
 - `sleep_debt_minutes`: SUM of ((`target_wake_time` - `target_bedtime`) - (`actual_wake_time` - `actual_bedtime`))
 - `avg_phone_pickups`: AVG of `phone_pickups`
 - `best_quality_day` / `worst_quality_day`: date with MAX/MIN `quality_rating`

Add routes/phone_pickups.rs:

Method	Path	Body / Params	Response	Auth?
POST	/phone-pickups	CreatePhonePickup	PhonePickup	Yes
GET	/phone-pickups	Query: ?sleep_id=UUID or ?session_id=UUID	Vec<PhonePickup>	Yes

1.4 — Sleep Android: Local Database

Room entity (`data/local/entity/SleepRecordEntity.kt`):

```
@Entity(tableName = "sleep_records")
data class SleepRecordEntity(
    @PrimaryKey val id: String,           // UUID as string
    val userId: String,
    val targetBedtime: String,           // "HH:mm" format
    val targetWakeTime: String,
    val actualBedtime: Long,             // epoch millis
    val actualWakeTime: Long,
    val qualityRating: Int,
    val phonePickups: Int,
    val totalPhoneMinutes: Int?,
    val notes: String?,
    val createdAt: Long,
    val updatedAt: Long,
    val isSynced: Boolean = false        // track sync state
)
```

Room DAO (`data/local/dao/SleepDao.kt`):

```
@Dao
interface SleepDao {
    @Query("SELECT * FROM sleep_records ORDER BY actualBedtime DESC")
    fun getAllRecords(): Flow<List<SleepRecordEntity>>

    @Query("SELECT * FROM sleep_records WHERE actualBedtime BETWEEN :start AND :end ORDER BY actualBedtime DESC")
    fun getRecordsBetween(start: Long, end: Long): Flow<List<SleepRecordEntity>>

    @Query("SELECT * FROM sleep_records WHERE id = :id")
    suspend fun getById(id: String): SleepRecordEntity?

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(record: SleepRecordEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(records: List<SleepRecordEntity>)

    @Delete
    suspend fun delete(record: SleepRecordEntity)

    @Query("SELECT * FROM sleep_records WHERE isSynced = 0")
    suspend fun getUnsyncedRecords(): List<SleepRecordEntity>

    @Query("UPDATE sleep_records SET isSynced = 1 WHERE id = :id")
    suspend fun markSynced(id: String)

    @Query("SELECT AVG(actualWakeTime - actualBedtime) FROM sleep_records WHERE actualBedtime BETWEEN :start AND :end")
    suspend fun avgDurationBetween(start: Long, end: Long): Double?

    @Query("SELECT AVG(qualityRating) FROM sleep_records WHERE actualBedtime BETWEEN :start AND :end")
}
```

```

suspend fun avgQualityBetween(start: Long, end: Long): Double?

    @Query("SELECT AVG(phonePickups) FROM sleep_records WHERE actualBedtime BETWEEN :start
AND :end")
    suspend fun avgPhonePickupsBetween(start: Long, end: Long): Double?
}

```

Add to `AppDatabase.kt`:

- Add `SleepRecordEntity` to `@Database(entities = [...])`
- Add abstract `fun sleepDao(): SleepDao`

1.5 — Sleep Android: API Service

Add to `ApiService.kt`:

```

@POST("sleep")
suspend fun createSleepRecord(@Body record: CreateSleepRecordDto): SleepRecordDto

@GET("sleep")
suspend fun getSleepRecords(
    @Query("start") start: String?,
    @Query("end") end: String?
): List<SleepRecordDto>

@GET("sleep/{id}")
suspend fun getSleepRecord(@Path("id") id: String): SleepRecordDto

@PUT("sleep/{id}")
suspend fun updateSleepRecord(
    @Path("id") id: String,
    @Body record: CreateSleepRecordDto
): SleepRecordDto

@DELETE("sleep/{id}")
suspend fun deleteSleepRecord(@Path("id") id: String)

@GET("sleep/stats")
suspend fun getSleepStats(
    @Query("range") range: String,
    @Query("start") start: String?,
    @Query("end") end: String?
): SleepStatsDto

```

DTOs (`data/remote/dto/SleepDtos.kt`):

- `CreateSleepRecordDto` — matches the backend's `CreateSleepRecord`
- `SleepRecordDto` — matches the backend's `SleepRecord` response
- `SleepStatsDto` — matches the backend's `SleepStats`
- Extension functions: `SleepRecordDto.toEntity()` and `SleepRecordEntity.toDto()`

1.6 — Sleep Android: Repository

`data/repository/SleepRepository.kt`:

```

class SleepRepository(
    private val sleepDao: SleepDao,
    private val apiService: ApiService
) {
    // Returns local data as Flow (UI always reads from local)
    fun getSleepRecords(): Flow<List<SleepRecordEntity>>

    // Returns records in date range
    fun getSleepRecordsBetween(start: Long, end: Long): Flow<List<SleepRecordEntity>>

    // Create: save locally first (offline-first), then push to server
    suspend fun createSleepRecord(record: CreateSleepRecordDto): Result<SleepRecordEntity>
        // 1. Generate UUID client-side
        // 2. Save to Room with isSynced = false
        // 3. Try to POST to API
        // 4. If success: mark isSynced = true
        // 5. If fail: leave as unsynced (will retry later)

    // Update: update locally, then push
    suspend fun updateSleepRecord(id: String, record: CreateSleepRecordDto):
    Result<SleepRecordEntity>

    // Delete: delete locally, then push
    suspend fun deleteSleepRecord(id: String): Result<Unit>

    // Sync: pull from server, merge into local DB
    suspend fun sync()
        // 1. Push all unsynced local records to server
        // 2. Pull all records from server
        // 3. Upsert into Room (server data wins on conflict)

    // Stats from local DB
    suspend fun getLocalStats(start: Long, end: Long): SleepStats
}

```

1.7 — Sleep Android: UI Screen

ui/sleep/SleepViewModel.kt:

State:

```

data class SleepUiState(
    val records: List<SleepRecordEntity> = emptyList(),
    val stats: SleepStats? = null,
    val isLoading: Boolean = false,
    val error: String? = null,
    val showAddDialog: Boolean = false,
    val selectedTimeRange: TimeRange = TimeRange.WEEK, // WEEK, MONTH
)

```

Functions:

- `loadRecords()` — observe `repository.getSleepRecords()` as Flow
- `loadStats(range)` — calculate from local data
- `addRecord(record)` — call `repository.createSleepRecord()`

- `deleteRecord(id)` — call `repository.deleteSleepRecord()`
- `sync()` — call `repository.sync()`

`ui/sleep/SleepScreen.kt`:

Layout (top to bottom):

1. **Header bar**: "Sleep Tracker" title + "Add" FAB (floating action button)
2. **Stats cards row** (horizontal scroll):
 - Card: "Avg Duration" — e.g. "7h 23m"
 - Card: "Avg Quality" — e.g. "3.8 / 5" with star icon
 - Card: "Sleep Debt" — e.g. "-2h 15m" (red if negative)
 - Card: "Avg Phone Pickups" — e.g. "2.3 / night"
3. **Bar chart**: sleep duration for each of the last 7 days (x-axis = day, y-axis = hours)
 - Bars colored by quality: green (4-5), yellow (3), red (1-2)
 - Horizontal line showing target sleep duration
4. **Tab row**: "Week" | "Month" toggle for stats/chart range
5. **Sleep history list** (`LazyColumn`):
 - Each item shows: date, duration (e.g. "7h 45m"), quality stars, phone pickups icon + count
 - Tap to expand: shows target vs actual times, notes, phone usage details
 - Swipe to delete

`ui/sleep/AddSleepDialog.kt`:

A bottom sheet or dialog with:

1. **Target bedtime** — time picker (defaults to user's usual target)
2. **Target wake time** — time picker
3. **Actual bedtime** — date + time picker
4. **Actual wake time** — date + time picker
5. **Quality rating** — row of 5 tappable stars
6. **Phone pickups** — number stepper (- / number / +)
7. **Total phone minutes** — optional number input
8. **Notes** — multiline text field
9. **Save button** — validates, calls `ViewModel`
10. **Cancel button**

Validation before save:

- Actual bedtime must be before actual wake time
- Quality rating must be selected (1-5)
- Phone pickups must be ≥ 0

1.8 — Sleep: Charting

Use Vico charting library.

Gradle dependency: `com.patrykandpatrick.vico:compose-m3` (latest)

Bar chart composable (`ui/sleep/SleepChart.kt`):

- X-axis: day labels (Mon, Tue, etc.)
- Y-axis: hours (0–12)
- Each bar = one night's sleep duration
- Bar color based on quality rating
- Horizontal dashed line = target duration

- Tap on bar to show tooltip with exact values

Phase 2: Pomodoro Sessions — End to End (Weeks 6–8)

2.1 — Sessions Backend: Database Table

Migration `003_create_sessions.sql`:

```
CREATE TABLE productivity_sessions (  
  id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  tag               VARCHAR(100) NOT NULL,  
  duration_minutes  INT NOT NULL,  
  work_duration     INT NOT NULL,          -- pomodoro work length in minutes  
  break_duration    INT NOT NULL,          -- pomodoro break length in minutes  
  phone_pickups     INT NOT NULL DEFAULT 0,  
  started_at        TIMESTAMPTZ NOT NULL,  
  ended_at          TIMESTAMPTZ,  
  completed         BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_sessions_user_id ON productivity_sessions(user_id);  
CREATE INDEX idx_sessions_started_at ON productivity_sessions(user_id, started_at);
```

Also add the FK to phone_pickups:

```
ALTER TABLE phone_pickups  
ADD CONSTRAINT fk_phone_pickups_session  
FOREIGN KEY (session_id) REFERENCES productivity_sessions(id) ON DELETE SET NULL;  
  
CREATE INDEX idx_phone_pickups_session ON phone_pickups(session_id);
```

2.2 — Sessions Backend: Models

`models/session.rs`:

```
struct ProductivitySession {  
  id: Uuid,  
  user_id: Uuid,  
  tag: String,  
  duration_minutes: i32,  
  work_duration: i32,  
  break_duration: i32,  
  phone_pickups: i32,  
  started_at: DateTime<Utc>,  
  ended_at: Option<DateTime<Utc>>,  
  completed: bool,  
  created_at: DateTime<Utc>,  
  updated_at: DateTime<Utc>,  
}
```

```
struct CreateSession {
    tag: String,
    work_duration: i32,      // e.g. 25
    break_duration: i32,    // e.g. 5
}

struct UpdateSession {
    ended_at: Option<DateTime<Utc>>,
    completed: Option<bool>,
    phone_pickups: Option<i32>,
    duration_minutes: Option<i32>,
}

struct SessionStats {
    total_focus_minutes_today: i64,
    total_focus_minutes_week: i64,
    sessions_completed_today: i64,
    sessions_completed_week: i64,
    current_streak_days: i64,      // consecutive days with at least 1 completed session
    longest_streak_days: i64,
    avg_phone_pickups_per_session: f64,
    total_phone_pickups_today: i64,
    top_tags: Vec<TagStat>,      // tag + total minutes, sorted by minutes desc
}

struct TagStat {
    tag: String,
    total_minutes: i64,
    session_count: i64,
}
```

2.3 — Sessions Backend: API Endpoints

routes/sessions.rs:

Method	Path	Body / Params	Response	Auth?
POST	/sessions	CreateSession	ProductivitySession	Yes
GET	/sessions	Query: ? start=DATE&end=DATE&tag=STRING	Vec<ProductivitySession>	Yes
GET	/sessions/:id	—	ProductivitySession	Yes
PUT	/sessions/:id	UpdateSession	ProductivitySession	Yes
DELETE	/sessions/:id	—	204 No Content	Yes
GET	/sessions/stats	Query: ?range=week\ month	SessionStats	Yes

POST /sessions logic:

- 1. Extract user ID
- 2. Set started_at = now(), completed = false, duration_minutes = 0
- 3. Insert and return

PUT /sessions/:id logic:

- 1. Verify session belongs to user

2. Update only the fields provided (partial update)
3. Set `updated_at = now()`
4. If `completed = true` and `ended_at` is not set, set `ended_at = now()`
5. Calculate `duration_minutes` from `ended_at - started_at`

GET /sessions/stats streak logic:

1. Query all distinct dates (from `started_at`) where user has at least 1 completed session, ordered DESC
2. Walk backwards from today: count consecutive days with no gaps → `current_streak_days`
3. Walk the full list: find longest consecutive run → `longest_streak_days`

2.4 — Sessions Android: Local Database

Room entity (`data/local/entity/SessionEntity.kt`):

```
@Entity(tableName = "productivity_sessions")
data class SessionEntity(
    @PrimaryKey val id: String,
    val userId: String,
    val tag: String,
    val durationMinutes: Int,
    val workDuration: Int,
    val breakDuration: Int,
    val phonePickups: Int,
    val startedAt: Long,
    val endedAt: Long?,
    val completed: Boolean,
    val createdAt: Long,
    val updatedAt: Long,
    val isSynced: Boolean = false
)
```

Room DAO (`data/local/dao/SessionDao.kt`):

- `getAllSessions(): Flow<List<SessionEntity>>`
- `getSessionsBetween(start, end): Flow<List<SessionEntity>>`
- `getById(id): SessionEntity?`
- `insert(session) / insertAll(sessions)`
- `update(session)`
- `delete(session)`
- `getUnsyncedSessions(): List<SessionEntity>`
- `markSynced(id)`
- `getCompletedSessionDates(start, end): List<Long>` — for streak calculation
- `getTotalFocusMinutes(start, end): Int?`

2.5 — Sessions Android: API & Repository

Add session endpoints to `ApiService.kt` — mirrors the backend routes.

`data/repository/SessionRepository.kt`:

- Same offline-first pattern as `SleepRepository`
- `createSession(tag, workDuration, breakDuration)` — creates locally, syncs to server
- `completeSession(id, durationMinutes, phonePickups)` — updates with final data

- `getActiveSessions()` — sessions where completed = false
- `sync()`

2.6 — Sessions Android: UI — Timer Screen

`ui/sessions/SessionsViewModel.kt:`

State:

```
data class SessionsUiState(
    val timerState: TimerState = TimerState.IDLE, // IDLE, RUNNING, PAUSED, BREAK, FINISHED
    val currentPhase: Phase = Phase.WORK, // WORK or BREAK
    val timeRemainingSeconds: Int = 0,
    val totalTimeSeconds: Int = 0,
    val workDuration: Int = 25, // minutes, user-configurable
    val breakDuration: Int = 5, // minutes, user-configurable
    val tag: String = "",
    val completedPomodoros: Int = 0, // count for current session
    val phonePickups: Int = 0,
    val todayStats: TodayStats? = null,
    val recentSessions: List<SessionEntity> = emptyList(),
    val error: String? = null,
)

data class TodayStats(
    val totalFocusMinutes: Int,
    val sessionsCompleted: Int,
    val currentStreak: Int,
    val phonePickupsToday: Int,
)
```

Functions:

- `startSession(tag)` — create session record, start countdown timer
- `pauseTimer()` / `resumeTimer()`
- `skipBreak()` — jump to next work phase
- `cancelSession()` — discard current session
- `completeSession()` — mark as completed, save to DB
- Timer logic: use `kotlinx.coroutines.delay(1000)` in a loop, decrement `timeRemainingSeconds`
- When work timer hits 0: auto-switch to break, increment `completedPomodoros`
- When break timer hits 0: auto-switch to work

`ui/sessions/SessionsScreen.kt:`

Layout (top to bottom):

1. **Today's stats bar** (horizontal row of mini stats):
 - Focus time today: "2h 15m"
 - Sessions: "4"
 - Streak: "5 days"
 - Distractions: "3 pickups"
2. **Timer circle** (center of screen, dominant visual):
 - Large circular progress indicator (Compose `Canvas` — draw arc)
 - Time remaining in center: "18:42"
 - Below circle: current phase label "WORK" or "BREAK"

- Below that: tag name (e.g. "LeetCode")

3. **Timer controls** (row of buttons):

- If IDLE: tag text field + work/break duration pickers + "Start" button
- If RUNNING: "Pause" button + "Cancel" button
- If PAUSED: "Resume" button + "Cancel" button
- If BREAK: "Skip Break" button

4. **Recent sessions list** (bottom section, [LazyColumn](#)):

- Each item: tag, duration, date, completed/cancelled, phone pickups
- Show last 10 sessions

[ui/sessions/TimerCircle.kt](#):

- Compose [Canvas](#) custom drawing
- Background circle (gray)
- Foreground arc (colored) representing progress: `sweepAngle = 360 * (timeRemaining / totalTime)`
- Color: work phase = blue/green, break phase = orange
- Animated smoothly using [animateFloatAsState](#)

2.7 — Sessions Android: Phone Pickup Detection

Uses Android [UsageStatsManager](#):

Add to [AndroidManifest.xml](#):

```
<uses-permission android:name="android.permission.PACKAGE_USAGE_STATS"
    tools:ignore="ProtectedPermissions" />
```

[util/PhoneUsageTracker.kt](#):

- Check if usage stats permission is granted: `AppOpsManager.checkOpNoThrow()`
- If not granted: show a prompt to open Settings → Apps → Special Access → Usage Access
- During an active session, poll `UsageStatsManager.queryEvents()` every 30 seconds
- Detect `MOVE_TO_FOREGROUND` events for non-system apps
- Each detected app open = one phone pickup
- Log pickups to local DB and increment counter in ViewModel

Phase 3: Calendar — End to End (Weeks 9–11)

3.1 — Calendar Backend: Database Table

Migration [004_create_calendar.sql](#):

```
CREATE TYPE event_category AS ENUM ('study', 'project', 'exercise', 'personal', 'other');
CREATE TYPE event_priority AS ENUM ('high', 'medium', 'low');

CREATE TABLE calendar_events (
    id                UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title             VARCHAR(255) NOT NULL,
    description        TEXT,
    start_time        TIMESTAMPTZ NOT NULL,
    end_time          TIMESTAMPTZ NOT NULL,
```

```
category      event_category NOT NULL DEFAULT 'other',
priority      event_priority NOT NULL DEFAULT 'medium',
is_recurring  BOOLEAN NOT NULL DEFAULT FALSE,
recurrence_rule TEXT,                -- e.g. "WEEKLY:MON,WED,FRI"
color         VARCHAR(7) NOT NULL DEFAULT '#4A90D9', -- hex color
created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_calendar_user_id ON calendar_events(user_id);
CREATE INDEX idx_calendar_start_time ON calendar_events(user_id, start_time);
```

3.2 — Calendar Backend: Models

models/calendar.rs:

```
struct CalendarEvent {
    id: Uuid,
    user_id: Uuid,
    title: String,
    description: Option<String>,
    start_time: DateTime<Utc>,
    end_time: DateTime<Utc>,
    category: EventCategory,      // enum: Study, Project, Exercise, Personal, Other
    priority: EventPriority,       // enum: High, Medium, Low
    is_recurring: bool,
    recurrence_rule: Option<String>,
    color: String,
    created_at: DateTime<Utc>,
    updated_at: DateTime<Utc>,
}

struct CreateCalendarEvent {
    title: String,
    description: Option<String>,
    start_time: DateTime<Utc>,
    end_time: DateTime<Utc>,
    category: EventCategory,
    priority: EventPriority,
    is_recurring: bool,
    recurrence_rule: Option<String>,
    color: Option<String>,        // defaults to category color if not provided
}
```

3.3 — Calendar Backend: API Endpoints

routes/calendar.rs:

Method	Path	Body / Params	Response	Auth?
POST	/calendar	CreateCalendarEvent	CalendarEvent	Yes
GET	/calendar	Query: ? start=DATE&end=DATE&category=STR&priority=STR	Vec<CalendarEvent>	Yes
GET	/calendar/:id	—	CalendarEvent	Yes

Method	Path	Body / Params	Response	Auth?
PUT	/calendar/:id	CreateCalendarEvent	CalendarEvent	Yes
DELETE	/calendar/:id	—	204 No Content	Yes

GET /calendar with recurrence expansion:

1. Query all events in date range (including recurring events whose `start_time` is before the range)
2. For each recurring event, parse `recurrence_rule`
3. Generate virtual event instances within the requested range
4. Merge one-time events + expanded recurring instances
5. Sort by `start_time` and return

Recurrence rule format: "WEEKLY:MON,WED,FRI" or "DAILY" or "MONTHLY:15" (day of month)

Expansion logic (`expand_recurrence`):

- Parse rule string
- Starting from event's `start_time`, step forward by rule interval
- For each step that falls within the query range, create a clone of the event with adjusted `start_time/end_time`
- Preserve the event duration (end - start stays the same)

3.4 — Calendar Android: Local Database

Room entity (`data/local/entity/CalendarEventEntity.kt`):

```
@Entity(tableName = "calendar_events")
data class CalendarEventEntity(
    @PrimaryKey val id: String,
    val userId: String,
    val title: String,
    val description: String?,
    val startTime: Long,
    val endTime: Long,
    val category: String,          // "study", "project", etc.
    val priority: String,         // "high", "medium", "low"
    val isRecurring: Boolean,
    val recurrenceRule: String?,
    val color: String,
    val createdAt: Long,
    val updatedAt: Long,
    val isSynced: Boolean = false
)
```

Room DAO: same pattern — CRUD + date range queries + unsynced queries.

3.5 — Calendar Android: Repository & API

Same offline-first pattern. Add calendar endpoints to `ApiService.kt`.

CalendarRepository:

- `getEventsForMonth(year, month): Flow<List<CalendarEventEntity>>`
- `getEventsForDay(date): Flow<List<CalendarEventEntity>>`
- `createEvent(event) / updateEvent(id, event) / deleteEvent(id)`

- `sync()`
- Client-side recurrence expansion for local data (mirror the backend logic)

3.6 — Calendar Android: UI

`ui/calendar/CalendarViewModel.kt:`

State:

```
data class CalendarUiState(
    val selectedDate: LocalDate = LocalDate.now(),
    val currentMonth: YearMonth = YearMonth.now(),
    val monthEvents: Map<LocalDate, List<CalendarEventEntity>> = emptyMap(),
    val selectedDayEvents: List<CalendarEventEntity> = emptyList(),
    val viewMode: ViewMode = ViewMode.MONTH, // MONTH or DAY
    val showAddDialog: Boolean = false,
    val isLoading: Boolean = false,
    val error: String? = null,
)
```

`ui/calendar/CalendarScreen.kt:`

Layout:

1. **Month header:** "< April 2026 >" with left/right arrows to change month
2. **Day-of-week header row:** Mon Tue Wed Thu Fri Sat Sun
3. **Month grid** (6 rows × 7 columns):
 - Each cell = one day
 - Current day highlighted with circle
 - Selected day highlighted with filled circle
 - Days with events show colored dots below the number (up to 3 dots, one per event category)
 - Tap a day → select it, load that day's events below
4. **Selected day section** (below grid):
 - Date heading: "Monday, April 12"
 - Event list for that day, sorted by start time:
 - Each event: colored left border (event color) + time range + title + category chip + priority indicator
 - Tap event → edit dialog
 - If no events: "No events" message
5. **FAB:** "+" to add new event

`ui/calendar/AddEventDialog.kt:`

Bottom sheet with:

1. **Title** — text field (required)
2. **Description** — multiline text field (optional)
3. **Start date + time** — date picker + time picker
4. **End date + time** — date picker + time picker
5. **Category** — dropdown/chip selector: Study, Project, Exercise, Personal, Other
6. **Priority** — segmented button: High, Medium, Low
7. **Color** — row of color circles to pick from
8. **Recurring toggle** — switch
 - If on: recurrence rule selector — frequency (Daily/Weekly/Monthly) + day picker for weekly
9. **Save button + Cancel button**

Validation:

- Title must not be empty
- Start time must be before end time
- If recurring, recurrence rule must be set

Use Kizitonwose Calendar library for the month grid — handles month scrolling, day selection, and custom day cell rendering.

Gradle dependency: `com.kizitonwose.calendar:compose` (latest)

Phase 4: Dashboard & Sync (Weeks 12–13)

4.1 — Dashboard Screen

`ui/dashboard/DashboardViewModel.kt`:

State:

```
data class DashboardUiState(  
    val lastNightSleep: SleepSummary? = null,  
    val todayFocus: FocusSummary? = null,  
    val upcomingEvents: List<CalendarEventEntity> = emptyList(),  
    val weeklyHighlights: WeeklyHighlights? = null,  
    val isLoading: Boolean = false,  
    val isSyncing: Boolean = false,  
)  
  
data class SleepSummary(  
    val duration: String,           // "7h 23m"  
    val quality: Int,              // 1-5  
    val vsTarget: String,          // "+30m" or "-1h 15m"  
    val phonePickups: Int,  
)  
  
data class FocusSummary(  
    val totalMinutesToday: Int,  
    val sessionsToday: Int,  
    val currentStreak: Int,  
    val phonePickupsToday: Int,  
)  
  
data class WeeklyHighlights(  
    val avgSleepDuration: String,  
    val avgSleepQuality: Double,  
    val totalFocusHours: Double,  
    val eventsCompleted: Int,  
    val eventsTotal: Int,  
)
```

`ui/dashboard/DashboardScreen.kt`:

Layout (scrollable column):

1. **Greeting header**: "Good morning, Oliver" (based on time of day)

2. **Sync indicator**: small "Last synced: 2 min ago" text + manual refresh button

3. **Last Night's Sleep card**:

- Big number: duration (e.g. "7h 23m")
- Quality stars
- vs target: green "+30m" or red "-1h 15m"
- Phone pickups: icon + count
- Tap → navigates to Sleep tab

4. **Today's Focus card**:

- Focus time: "2h 15m"
- Sessions: "4 completed"
- Streak: "5 days" with flame icon
- Phone distractions: count
- Tap → navigates to Sessions tab

5. **Upcoming Events section**:

- Next 5 events from calendar, each showing: time + title + category color
- Tap event → navigates to Calendar tab with that day selected
- "View all" link → Calendar tab

6. **Quick Actions row** (3 buttons):

- "Log Sleep" → opens add sleep dialog
- "Start Focus" → navigates to Sessions with timer ready
- "Add Event" → opens add event dialog

7. **Weekly Highlights card** (summary of the week):

- Avg sleep: duration + quality
- Total focus: hours
- Events: completed/total

4.2 — Sync System

data/repository/SyncManager.kt:

Coordinates sync across all repositories:

```
class SyncManager(  
    private val sleepRepo: SleepRepository,  
    private val sessionRepo: SessionRepository,  
    private val calendarRepo: CalendarRepository,  
) {  
    // Full sync: push unsynced local data, pull remote data  
    suspend fun syncAll(): Result<Unit> {  
        return runCatching {  
            // Push all unsynced records first  
            sleepRepo.pushUnsynced()  
            sessionRepo.pushUnsynced()  
            calendarRepo.pushUnsynced()  
  
            // Then pull latest from server  
            sleepRepo.pullFromServer()  
        }  
    }  
}
```

```

        sessionRepo.pullFromServer()
        calendarRepo.pullFromServer()
    }
}

// Triggered on:
// 1. App launch (after auth check passes)
// 2. Manual pull-to-refresh on dashboard
// 3. Periodic background sync via WorkManager (every 30 min when app is in background)
}

```

Background sync with WorkManager:

```

class SyncWorker(context: Context, params: WorkerParameters) : CoroutineWorker(context,
params) {
    override suspend fun doWork(): Result {
        // Get SyncManager instance
        // Call syncAll()
        // Return Result.success() or Result.retry()
    }
}

// Schedule in Application.onCreate():
// PeriodicWorkRequestBuilder<SyncWorker>(30, TimeUnit.MINUTES).build()

```

4.3 — Pull-to-Refresh on Dashboard

- Wrap dashboard content in `SwipeRefreshLayout` (from Accompanist or Material3 pull-to-refresh)
- On refresh: call `SyncManager.syncAll()`, then reload all dashboard data
- Show `isSyncing` indicator during sync

Phase 5: Polish & Advanced Features (Weeks 14–16)

5.1 — Notifications & Reminders

`util/NotificationHelper.kt`:

- Create notification channel: "Reminders" (ID: `reminders`)
- Helper function to show a notification with title, body, and tap action (deep link to specific screen)

Bedtime reminder:

- User sets their target bedtime (stored in DataStore preferences)
- Schedule with `AlarmManager.setExactAndAllowWhileIdle()` for 30 minutes before target bedtime
- Notification: "Time to wind down — bedtime in 30 minutes"
- `BroadcastReceiver` to fire the notification

Session reminder:

- User can set a daily "focus time" reminder (e.g. "2:00 PM")
- Same AlarmManager pattern
- Notification: "Time to focus! Start a session?"
- Tap → opens app on Sessions tab

Calendar event reminders:

- When creating/updating an event, schedule a reminder 15 min before `start_time`
- Use `AlarmManager` with the event ID as extra
- Notification: "In 15 min: {event title}"
- Tap → opens app on Calendar tab with that day

Morning summary notification:

- Schedule daily at 8:00 AM (configurable)
- Content: "You slept {duration}. You have {count} events today."
- Query last night's sleep record + today's calendar events

5.2 — UI Theming & Polish

Theme system:

- `settings/SettingsScreen.kt` with theme toggle: Light / Dark / System
- Store preference in `DataStore`
- Apply in `Theme.kt` using `isSystemInDarkTheme()` + override

Material 3 theme:

- Define color schemes for light and dark in `Color.kt`
- Primary: a calm blue-purple
- Category colors: Study = blue, Project = green, Exercise = orange, Personal = purple, Other = gray
- Priority colors: High = red, Medium = yellow, Low = green

Animations:

- Screen transitions: slide in/out using Compose Navigation animations
- Timer: smooth arc animation via `animateFloatAsState`
- Stats cards: fade-in on load
- List items: slide-in animation using `AnimatedVisibility`

Onboarding flow (`ui/onboarding/OnboardingScreen.kt`):

- Screen 1: "Welcome" — app description + continue
- Screen 2: "Set Your Sleep Target" — bedtime + wake time pickers
- Screen 3: "Focus Preferences" — default work/break duration
- Screen 4: "All Set!" — done button
- Store `onboarding_completed = true` in `DataStore`
- Show only on first launch (before login/register)

Empty states:

- Sleep: illustration + "No sleep records yet. Log your first night!"
- Sessions: illustration + "Ready to focus? Start your first session!"
- Calendar: illustration + "No events today. Tap + to create one."
- Dashboard: show all sections with placeholder/empty content

5.3 — Weekly Reports & Streaks

`ui/reports/WeeklyReportScreen.kt`:

Accessed from dashboard or a "Reports" button in settings/nav.

Layout (scrollable):

- 1. **Week selector:** "< Apr 6 – Apr 12 >" with arrows
- 2. **Sleep section:**
 - Bar chart: 7 bars for daily sleep duration
 - Avg duration, avg quality, best/worst night
 - Phone usage during sleep: avg pickups/night, trend arrow (up/down vs last week)
- 3. **Focus section:**
 - Bar chart: 7 bars for daily focus hours
 - Total hours, sessions completed, top 3 tags with time breakdown
 - Phone distractions: total pickups, avg per session, best distraction-free streak
- 4. **Calendar section:**
 - Events completed vs total
 - Busiest day
 - Category breakdown (pie chart or horizontal bars)
- 5. **Streaks & Achievements section:**
 - Current sleep target streak: "5 nights" (hitting target bedtime/duration)
 - Current focus streak: "8 days" (at least 1 completed session/day)
 - Distraction-free sessions streak: "3 sessions" (0 phone pickups)
 - Milestones list: show earned achievements with icons

Achievement definitions (hardcoded list):

Achievement	Condition
"Early Bird"	7 consecutive days hitting target wake time
"Night Owl No More"	7 consecutive days hitting target bedtime
"Focus Master"	50 total focus hours
"Century"	100 completed sessions
"Zen Mode"	5 sessions with 0 phone pickups
"Iron Streak"	30-day focus streak
"Sleep Champion"	14-day sleep target streak
"Marathon"	4+ hours focus in a single day

Store earned achievements in Room table: `achievements(id, name, earned_at)`. Check conditions after each sleep record save and session complete.

5.4 — Settings Screen

`ui/settings/SettingsScreen.kt`:

Options:

- **Sleep targets:** target bedtime, target wake time, target duration
- **Focus defaults:** work duration (default 25), break duration (default 5)
- **Notifications:** toggle each reminder type on/off, set times
- **Theme:** Light / Dark / System
- **Account:** email display, logout button
- **About:** app version

All preferences stored in DataStore.

Phase 6: Deployment (Weeks 17–18)

6.1 — Backend Deployment

Dockerize the backend:

backend/Dockerfile:

```
FROM rust:1.77 AS builder
WORKDIR /app
COPY . .
RUN cargo build --release

FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y ca-certificates && rm -rf /var/lib/apt/lists/*
COPY --from=builder /app/target/release/backend /usr/local/bin/backend
EXPOSE 8080
CMD ["backend"]
```

Production docker-compose.yml:

```
services:
  backend:
    build: ./backend
    ports:
      - "8080:8080"
    environment:
      DATABASE_URL: ${DATABASE_URL}
      JWT_SECRET: ${JWT_SECRET}
    depends_on:
      - db

  db:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Deploy to a VPS (Fly.io or Railway):

1. Create account on platform
2. Install CLI tool
3. Deploy using platform-specific commands
4. Set environment variables via platform dashboard
5. Set up HTTPS (platform-provided TLS)
6. Verify all endpoints work via curl

Update Android app:

- Change `RetrofitClient.kt` base URL from `http://10.0.2.2:8080` to `https://your-production-url.com`
- Use `BuildConfig` to switch between debug (local) and release (production) URLs

6.2 — Google Play Store Release

Generate signed AAB:

1. Android Studio → Build → Generate Signed Bundle/APK
2. Create a new keystore (save it securely — you need this for all future updates)
3. Build release AAB

Google Play Console:

1. Create Google Play Developer account (\$25 one-time)
2. Create new app
3. Fill in store listing:
 - App name: "Ultimate Productivity"
 - Short description (80 chars)
 - Full description
 - Screenshots: at least 2 phone screenshots per screen (dashboard, sleep, sessions, calendar)
 - App icon: 512x512 PNG
 - Feature graphic: 1024x500
4. Content rating questionnaire
5. Privacy policy (required — host a simple page on GitHub Pages or similar)
6. Set pricing: Free
7. Upload AAB to Production track
8. Submit for review

Phase Summary

Phase	Weeks	What You'll Have
Phase 0: Scaffold & Auth	1–2	Both projects running, user login/register working end to end
Phase 1: Sleep Tracking	3–5	Full sleep tracker: API + Android screen + charts + sync
Phase 2: Pomodoro Sessions	6–8	Full pomodoro timer: API + Android screen + streaks + phone detection
Phase 3: Calendar	9–11	Full calendar: API + Android screen + recurring events
Phase 4: Dashboard & Sync	12–13	Dashboard home screen, background sync across all features
Phase 5: Polish	14–16	Notifications, theming, weekly reports, achievements, settings
Phase 6: Deploy	17–18	Backend on VPS, app on Google Play Store
Total	~18 weeks	Production app on Google Play

Assumes ~15–20 hours/week. Adjust based on your availability around uni.